Laboratorium nr 5

Olekasndr Melnyk

Wskazówki i dobre praktyki podczas tworzenia plików Dockerfile. Automatyczna kontrola poprawności działania aplikacji, Budowa obrazów metodą „od podstaw. Wieloetapowe budowanie obrazów

```
oleksandrmelnik@Oleksandrs-MacBook-Pro lab5 % docker build --build-arg VERSION=1.2.3-golang -t lab5-app-go .
[+] Building 2.9s (13/13) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 1.21kB                                                                              0.0s
 => [internal] load metadata for docker.io/library/golang:1.21-alpine                                               0.6s
 => [internal] load metadata for docker.io/library/nginx:alpine                                                     0.6s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [builder 1/4] FROM docker.io/library/golang:1.21-alpine@sha256:2414035b086e3c42b99654c8b26e6f5b1b1598080d65fd  0.0s
 => => resolve docker.io/library/golang:1.21-alpine@sha256:2414035b086e3c42b99654c8b26e6f5b1b1598080d65fd03c7f499  0.0s
 => CACHED [stage-1 1/3] FROM docker.io/library/nginx:alpine@sha256:e7257f1ef28ba17cf7c248cb8ccf6f0c6e0228ab9c315  0.0s
 => => resolve docker.io/library/nginx:alpine@sha256:e7257f1ef28ba17cf7c248cb8ccf6f0c6e0228ab9c315c152f9c203cd34c  0.0s
 => [internal] preparing inline document                                                                            0.0s
 => CACHED [builder 2/4] WORKDIR /app                                                                               0.0s
 => [builder 3/4] COPY <<EOF main.go                                                                                0.0s
 => [builder 4/4] RUN CGO_ENABLED=0 GOOS=linux go build -o html-gen main.go                                         2.0s
 => [stage-1 2/3] COPY --from=builder /app/html-gen /usr/local/bin/html-gen                                         0.0s
 => [stage-1 3/3] RUN echo '#!/bin/sh' > /entrypoint.sh &&     echo '/usr/local/bin/html-gen > /usr/share/nginx/h  0.1s
 => exporting to image                                                                                              0.1s
 => => exporting layers                                                                                             0.1s
 => => exporting manifest sha256:71095aa5de164766bd460d6e4f6a218729a5fce3fa425a4040374064e580fbb3                   0.0s
 => => exporting config sha256:4745fa565b3c15cd96943b0d3c247b9235efb990843f31d075af379f358cf17d                     0.0s
 => => exporting attestation manifest sha256:2706b2846aa7d80ea60b10fd66491c2f6648b950d4af120844d7c8917fa3870f       0.0s
 => => exporting manifest list sha256:d4b9b7f9ecca349b9233e2bc500dc781454da213c475851058274c8875ec9508              0.0s
 => => naming to docker.io/library/lab5-app-go:latest                                                               0.0s
 => => unpacking to docker.io/library/lab5-app-go:latest                                                            0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ilvagwo43dqszhbcmc3v4277q
```

To polecenie służy do wytworzenia nowego obrazu Dockera przy ścisłym zachowaniu instrukcji zawartych w pliku Dockerfile.

```
oleksandrmelnik@Oleksandrs-MacBook-Pro lab5 % docker run -d -p 8080:80 --name app  lab5-app-go
26dcb0f9a7d2f54a1a5c8dd3e88df9ed97a9f39949c7dc99bb6ce4ee2f81b536
```

Wykonanie tego polecenia skutkuje powołaniem do życia aktywnego kontenera, który powstaje na fundamencie wcześniej przygotowanego obrazu o nazwie lab5-app-go. Zastosowanie flagi -d sprawia, że proces przechodzi w tryb pracy w tle, co umożliwia swobodne korzystanie z konsoli, podczas gdy serwer stabilnie operuje pod spodem. Parametr -p 8080:80 pełni funkcję mostu sieciowego, przekierowując ruch przychodzący na port 8080 komputera hosta bezpośrednio na port 80 wewnątrz kontenera. Dodatkowo opcja --name app nadaje instancji czytelną nazwę, co znacząco upraszcza późniejsze operacje administracyjne i eliminuje potrzebę posługiwania się długimi ciągami znaków ID.

```
oleksandrmelnik@Oleksandrs-MacBook-Pro lab5 % docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                   PORTS                                       NAMES
26dcb0f9a7d2   lab5-app-go   "/docker-entrypoint.…"   8 minutes ago   Up 8 minutes (healthy)   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   app
oleksandrmelnik@Oleksandrs-MacBook-Pro lab5 %
```

Polecenie docker ps służy do generowania zestawienia wszystkich aktualnie uruchomionych kontenerów wraz z ich podstawowymi parametrami technicznymi. Wyświetlony w kolumnie statusu komunikat Up 10 seconds (health: starting) informuje, że instancja została poprawnie włączona, a zaimplementowany mechanizm HEALTHCHECK rozpoczął właśnie wstępną fazę sprawdzania kondycji usług.

```
oleksandrmelnik@Oleksandrs-MacBook-Pro lab5 % curl http://localhost:8080
<html><body><h1>Laboratorium 5 - Go + Nginx</h1><p><b>Wersja:</b> %s</p>%!(EXTRA string=1.2.3-golang)<p><b>Hostname:</b> %s</p>%!(EXTRA string=26dcb0f9a7d2)<p><b>IP:</b> %s</p>%!(EXTRA string=172.17.0.2)</bo
```

Wykorzystanie polecenia curl http://localhost:8080 pozwala na wysłanie bezpośredniego zapytania HTTP pod wskazany adres lokalny i wyświetlenie otrzymanej odpowiedzi tekstowej bezpośrednio w oknie terminala.